



KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Hafta 2 Laboratuvar Föyü

HTML5 ve CSS3: Statik Etkinlik Sayfasını Kurun

Doç. Dr. Tolga Berber · <https://avesis.ktu.edu.tr/tberber>

🕒 Tahmini süre:	2 saat
💻 Gereksinimler:	Güncel bir tarayıcı (Chrome/Firefox/Edge), bir metin editörü (VS Code önerilir), isteğe bağlı Python 3.12+ (http.server için) — npm/derleme aracı gerekmez

Öğrenme fazları: 1 • Ön-öğretim 2 • İçerik 3 • Katılım 4 • Değerlendirme 5 • İzleme

1 • Ön-öğretim 📣 Neden Önemli?

Geçen hafta `curl` ile gerçek bir sunucuya sorup ham bir HTML metnini gördünüz (Hf. 1 Ünite 2 — statik içerik); ama o metni siz yazmadınız, sunucu diskte hazır duran bir dosyayı olduğu gibi gönderdi. Bugün o dosyanın **ta kendisini**, sıfırdan, kendi ellerinizle yazıyorsunuz — tarayıcının okuyup ekrana çizdiği gerçek bir HTML5 + CSS3 sayfası. İki saat sonra elinizde Etkinlik/Bilet Sisteminin ilk gerçek ekranı olacak: gezinme menülü bir etkinlik listesi, bir koltuk planı ve bir bilet formu — hepsi hem cep telefonunda hem masaüstünde düzgün görünecek.

1 • Ön-öğretim 🔄 Ön Bilgi

Bu föy Hafta 2 ders notunun dört ünitesine dayanır: Ünite 1 (semantik HTML: belge yapısı, form, tablo), Ünite 2 (CSS temelleri: seçiciler, kaskad, kutu modeli, birimler), Ünite 3 (Flexbox ve Grid ile yerleşim) ve Ünite 4 (duyarlı tasarım ve erişilebilirlik). Hafta 1 Ünite 2'nin “statik içerik” kavramını (sunucunun diskteki bir dosyayı olduğu gibi göndermesi) ve Ünite 3'ün “sunum katmanı” kavramını (kullanıcının gördüğü/etkileşime girdiği arayüz) hazır bilgi olarak kullanacaksınız; bu föyde React henüz yok, ama React'in Hf. 4'te üzerine oturacağı yapı ve görünüm bugün kuruluyor.

Föyü şöyle kullanın: her adımda önce *Ne yapacağız?* kutusundaki görevi kendi dosyalarınıza **elle yazarak** tamamlayın, sonra sayfayı tarayıcıda yenileyip gördüğünüzü *Beklenen sonuç* kutusuyla karşılaştırın. Adımlardaki kod kutuları `kod_ornekleri/hafta02/` klasöründeki referans dosyalardan gelir — bunlar hedef/“doğru cevap” niteliğindedir; siz bu kodu kopyala-yapıştır yapmak yerine kendi `web-lab/` klasörünüzdeki dosyalara yazarsınız, sonucu tarayıcıda karşılaştırırsınız. Adımlar ders notunun dört ünitesiyle aynı sırada ilerler; her bölümün başında ilgili üniteye bir atıf bulacaksınız. Birkaç adımda bir gelen *Kontrol Noktası*, o ana kadar yazdıklarınızı bir arada toparlamanıza yarar.

1 Semantik HTML: Belge Yapısını Kurun

(bkz. Ders notu Hafta 2, Ünite 1 — Semantik HTML: Belge Yapısı, Form ve Tablo.)

Adım 1: Proje klasörünüzü ve boş dosyalarınızı hazırlayın

? Ne yapacağız? Bilgisayarınızda web-lab adlı boş bir klasör açıp içine üç boş dosya — index.html, etkinlik-detay.html, stiller.css — oluşturacaksınız.

i Neden? Bu hafta yazacağınız dosyalar npm/derleme aracı gerektirmez — tarayıcı bu dosyaları doğrudan diskten okur (Hf. 1 Ünite 2 “statik içerik”). Üç dosyayı en baştan hazırlamak, ilerleyen adımlarda doğru kodu doğru dosyaya yazdığınızdan emin olmanızı sağlar.

```
>_ Terminal
$ mkdir web-lab && cd web-lab
$ touch index.html etkinlik-detay.html stiller.css
$ ls
```

👁 Beklenen sonuç: web-lab klasöründe üç dosya görünür, hepsi 0 bayt boyutunda.

```
🖥 Beklenen Çıktı
etkinlik-detay.html
index.html
stiller.css
```

🖥 macOS / Linux

Yukarıdaki touch komutu her iki sistemde de hazır gelir.

🖥 Windows

PowerShell’de sırasıyla:

- mkdir web-lab; cd web-lab
- ni index.html
- ni etkinlik-detay.html
- ni stiller.css

komutlarıyla aynı üç dosyayı oluşturun.

Adım 2: index.html dosyasına belge iskeletini yazın

? Ne yapacağız? index.html dosyanıza aşağıdaki belge iskeletini elle yazacaksınız: <!DOCTYPE html>, <html lang="tr">, <head> içinde karakter kümesi, viewport meta etiketi ve başlık, en sonda stiller.css dosyasına bir bağlantı.

i Neden? <meta charset="UTF-8"> olmadan Türkçe karakterleriniz (ğ, ş, ı) tarayıcıda bozuk görünür. viewport meta etiketi ise şimdilik görünmez bir etkiye sahip gibi dursa da Ünite 4’te öğreneceğiniz duyarlı tasarımın **önkoşuludur** — mobil tarayıcının sayfayı masaüstü genişliğinde küçültmesini engeller. Bu yüzden onu en baştan, unutmadan ekliyoruz.

```
1 <!DOCTYPE html>
2 <html lang="tr">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1">
6   <title>KTÜ Etkinlik ve Bilet Sistemi</title>
```

Adım 2 (devamı)

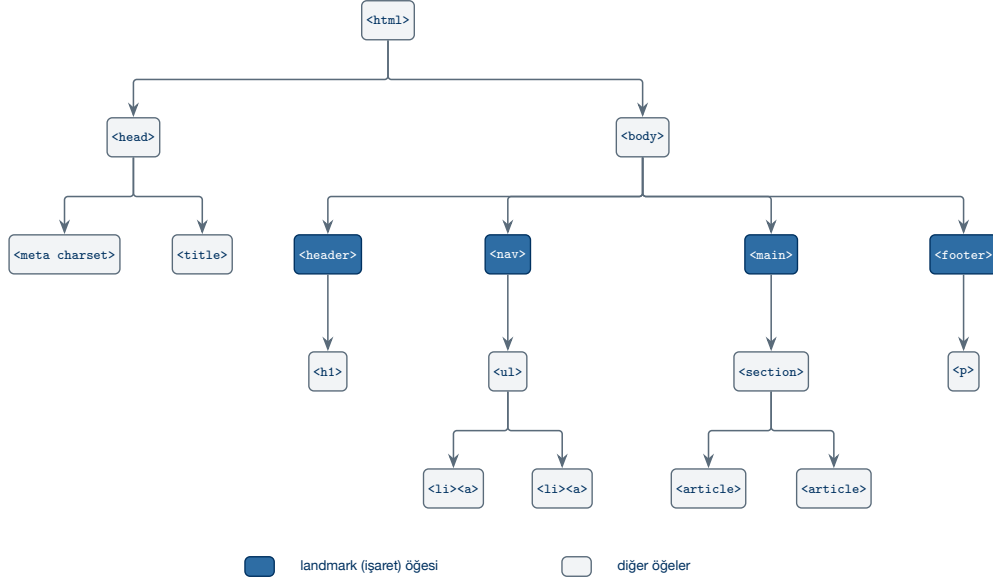
```
7 <link rel="stylesheet" href="stiller.css">
8 </head>
```

Beklenen sonuç: Dosyayı tarayıcıda açtığınızda sekme başlığında “KTÜ Etkinlik ve Bilet Sistemi” yazısını görürsünüz; sayfa gövdesi henüz boş olduğundan içerik alanı beyaz kalır.

Adım 3: Semantik landmark öğeleriyle sayfa iskeletini kurun

Ne yapacağız? `<body>` içine, bir önceki adımda kapattığınız `</head>` etiketinin hemen ardından, `<header>` içinde bir `<nav>` menüsü ve onu izleyen boş bir `<main>` etiketi yazacaksınız (`<main>` etiketini bu adımda açık bırakın — içeriğini sonraki adımlarda dolduracaksınız).

Neden? Sayfanın tamamını `<div>` ile yazmak da tarayıcıda aynı görünürdü — ama bir ekran okuyucu ya da arama motoru botu, hangi `<div>`’in menü, hangisinin ana içerik olduğunu **anlayamaz**. `<header>/<nav>/<main>/<footer>` gibi landmark öğeleri, aynı görünümü korurken koda **anlam** ekler.



```
1 <header>
2   <nav class="site-nav">
3     <ul>
4       <li><a href="index.html">Etkinlikler</a></li>
5       <li><a href="etkinlik-detay.html">Bilet Al</a></li>
6     </ul>
7   </nav>
8 </header>
9
10 <main>
```

Beklenen sonuç: Tarayıcıda menüde “Etkinlikler” ve “Bilet Al” bağlantıları görünür; ikisi de henüz stilsiz, alt alta, mavi ve altı çizili bağlantılar gibi durur — bu normaldir, CSS’i Bölüm 2’de ekleyeceksiniz.

Adım 4: Başlık hiyerarşisini ve etkinlik kartlarını ekleyin

❓ Ne yapacağız? <main> içine sırasıyla bir <h1> sayfa başlığı, kısa bir tanıtım paragrafı, bir <h2> alt başlık ve bir <section> içinde etkinlik kartlarını (article) elle yazacaksınız. Referans dosyada **tek bir** kart etiketlenmiştir; siz aynı deseni — farklı başlık/tarih/mekân bilgileriyle — kopyala değil, elle tekrar ederek **iki kart daha** ekleyin (toplam 3 kart).

i Neden? Başlık hiyerarşisinde tek bir <h1> bulunması ve alt başlıkların atlama yapmadan (h2'den sonra h3, doğrudan h4'e değil) ilerlemesi, hem ekran okuyucuların sayfada gezinme listesi çıkarabilmesi hem de arama motorlarının içeriğin önem sırasını doğru okuyabilmesi için gereklidir.

```
1 <h1>KTÜ Etkinlik ve Bilet Sistemi</h1>
2 <p>Kampüsteki güncel etkinlikleri inceleyin, dilediğiniz etkinlik için bilet almaya
  başlayın.</p>
3
4 <h2>Öne Çıkan Etkinlikler</h2>

1 <article class="etkinlik-karti">
2   
3   <h3>Bahar Konseri</h3>
4   <p>15 Mayıs 2026 &mdot; Açık Hava Amfisi</p>
5   <p>Üniversite korosunun ve konuk sanatçıların katılımıyla bahar konseri.</p>
6   <a href="etkinlik-detay.html">Detaylar ve bilet</a>
7 </article>
```

👁 Beklenen sonuç: Sayfada bir başlık, bir paragraf, “Öne Çıkan Etkinlikler” alt başlığı ve altında (henüz stilsiz, alt alta dizili) 3 etkinlik kartı görünür; her kartta bir resim yer tutucusu, kart başlığı, tarih/ mekân bilgisi ve bir “Detaylar ve bilet” bağlantısı bulunur.

💡 İpucu

Resim adresi olarak referans dosyadaki <https://via.placeholder.com/320x180> gibi bir yer tutucu kullanabilirsiniz; alt özneteliğini her zaman resmin ne gösterdiğini anlatacak biçimde (boş bırakmadan) yazın.

Adım 5: Etkinlik listesi tablosunu ekleyin

❓ Ne yapacağız? İkinci <h2> alt başlığının (“Etkinlik Listesi”) altına, <table>/<thead>/<tbody> yapısıyla başlık, tarih, mekân ve kapasite sütunlarından oluşan bir tablo yazacak, ardından </main>, <footer> ve kapanış etiketleriyle sayfayı tamamlayacaksınız.

i Neden? Bir tablo, satır ve sütunları anlamsal olarak birbirine bağlı veriler (ör. her satır bir etkinlik, her sütun o etkinliğin bir özelliği) için doğru araçtır — <div> satırlarıyla aynı görünümü elde etmeye çalışmak, ekran okuyucunun “bu bir tablodur, 4 sütunu var” bilgisini kaybetmesine yol açar.

```
1 <table>
2   <thead>
3     <tr>
4       <th scope="col">Başlık</th>
5       <th scope="col">Tarih</th>
6       <th scope="col">Mekân</th>
7       <th scope="col">Kapasite</th>
8     </tr>
9   </thead>
```

Adım 5 (devamı)

```

10 <tbody>
11 <tr>
12 <td>Bahar Konseri</td>
13 <td>15.05.2026</td>
14 <td>Açık Hava Amfisi</td>
15 <td>500</td>
16 </tr>
17 <tr>
18 <td>Kariyer Günleri</td>
19 <td>22.05.2026</td>
20 <td>Kongre Merkezi</td>
21 <td>800</td>
22 </tr>
23 <tr>
24 <td>Tiyatro Gecesi</td>
25 <td>29.05.2026</td>
26 <td>Kültür Merkezi</td>
27 <td>250</td>
28 </tr>
29 </tbody>
30 </table>

```

👁 **Beklenen sonuç:** Kartların altında, başlık satırı diğerlerinden görsel olarak ayrışan (henüz stilsiz de olsa tarayıcının varsayılan tablo çizgileriyle) 3 satırlık bir etkinlik tablosu görünür.

✅ **Kontrol Noktası — devam etmeden doğrulayın**

index.html dosyanızı tarayıcınızda açıp (dosyaya çift tıklayarak ya da python3 -m http.server ile) menü bağlantılarını, 3 etkinlik kartını ve etkinlik tablosunu gördüyseniz, belge yapısını doğru kurmuşsunuz demektir — CSS'e geçebilirsiniz.

💡 **İpucu**

Dosyayı çift tıklamak yerine aynı klasörde python3 -m http.server çalıştırıp http://localhost:8000/index.html adresini açarsanız, Hafta 1'de curl ile aldığınız cevabın aynı statik dosyanın bu kez gerçek bir HTTP isteği/cevabı üzerinden geldiğini de gözlemlemiş olursunuz.

2 CSS Temelleri: Renk, Tipografi ve Kutu Modeli

(bkz. Ders notu Hafta 2, Ünite 2 — CSS Temelleri: Seçiciler, Kaskad, Kutu Modeli, Birimler.)

Adım 6: stiller.css dosyasına renk ve tipografi tanımlarını yazın

❓ **Ne yapacağız?** stiller.css dosyanızın en başına, :root içinde KTÜ paletinin özel özellik (custom property) tanımlarını, hemen ardından body için font ailesi, punto ve metin rengini yazacaksınız. index.html zaten 2. adımda bu dosyaya bağlandığından ekstra bir bağlantı eklemenize gerek yoktur.

❗ **Neden?** Renkleri her seferinde tek tek yazmak yerine :root içinde bir kez tanımlayıp var(--renk-lacivert) ile çağırmak, aynı rengi 10 farklı yerde kullandığınızda tek satırda değiştirebilmenizi sağlar — bu, kaskadın (CSS kurallarının birleşerek uygulanması) pratikte

Adım 6 (devamı)

size sağladığı ilk kolaylıktır.

```

1 :root {
2   --renk-lacivert: #003160;
3   --renk-mavi: #2E6DA4;
4   --renk-acikmavi: #E8F0F7;
5   --renk-gri: #5A6B7A;
6   --renk-acikgri: #F2F4F6;
7   --renk-yesil: #2E7D52;
8   --renk-kehribar: #B0681B;
9 }
10
11 body {
12   font-family: "Segoe UI", Helvetica, Arial, sans-serif;
13   font-size: 1rem;
14   line-height: 1.5;
15   color: var(--renk-lacivert);
16   background-color: #FFFFFF;
17 }

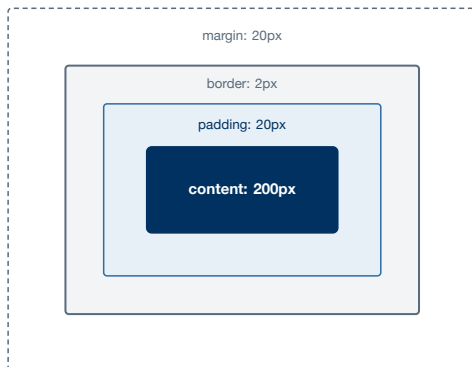
```

👁 **Beklenen sonuç:** Sayfayı yenilediğinizde gövde metninin rengi lacivert (#003160) olarak değişir; henüz başka bir görsel fark yoktur — kutu modeli ve yerleşim kuralları sonraki adımlarda gelecektir.

Adım 7: Kutu modelini sıfırlayın ve genişlik farkını elle doğrulayın

❓ **Ne yapacağız?** stiller.css dosyanıza aşağıdaki kutu modeli sıfırlama kuralını ekleyeceksiniz. Ardından — yalnızca bu deney için — dosyanın en sonuna geçici bir test kuralı olan `.test-kutu { width: 200px; padding: 20px; border: 2px solid var(--renk-kehribar); }` kuralını, `index.html`'e de `<main>` içine geçici bir `<div class="test-kutu">Test kutusu</div>` satırı ekleyip tarayıcınızın geliştirici araçlarıyla ("Elemanı İncele") kutunun toplam genişliğini ölçeceksiniz.

📌 **Neden?** `box-sizing: content-box` (tarayıcı varsayılanı) iken toplam genişlik = içerik + $2 \times \text{padding} + 2 \times \text{border}$ = $200 + 40 + 4 = 244\text{px}$ olur — `width: 200px` yazmanıza rağmen kutu daha geniş görünür. `box-sizing: border-box` eklendiğinde padding ve border artık tanımladığınız genişliğin içine sayılır, toplam genişlik tam **200px**'te sabit kalır. Bu, Ünite 2'nin en sık karşılaşılan "gizemli genişlik" hatasının çözümüdür.



box-sizing: content-box
toplam genişlik =
 $200 + 2 \times 20 + 2 \times 2 = 244\text{px}$

box-sizing: border-box
toplam genişlik =
200px (sabit)

```

1 * {
2   box-sizing: border-box;
3 }
4
5 body {

```

Adım 7 (devamı)

```

6  margin: 0;
7  }
8
9  img {
10 max-width: 100%;
11 height: auto;
12 }

```

👁 **Beklenen sonuç:** kutu_sifirlama kuralını eklemeyen önce test kutusunun toplam genişliği DevTools'ta **244px** görünür; kuralı ekledikten sonra aynı kutu **200px**'e düşer. Deneyi bitirince .test-kutu kuralını ve index.html'deki geçici <div> satırını silin — bunlar yalnızca gözlem içindi, kalıcı sayfanın bir parçası değildir.

Ortak Hata / Tuzak

* { box-sizing: border-box; } kuralını dosyanın **sonuna** eklemek de tarayıcıda çalışır (kaskad son kuralı uygular) ama alışkanlık olarak bu tür genel sıfırlama kural-larını dosyanın **en başında** tutmak, sonraki geliştiricilerin (ve gelecekteki sizin) kuralı aramadan bulmasını sağlar.

✓ Kontrol Noktası — devam etmeden doğrulayın

Sayfanızda gövde metninin lacivert göründüğünü ve test -kutu deneyinde border -box ile toplam genişliğin 244px'ten 200px'e düştüğünü DevTools'ta gördüyseniz, kutu modelini doğru kavramışsınız demektir — yerleşime geçebilirsiniz.

3 Yerleşim: Flexbox ve Grid

(bkz. Ders notu Hafta 2, Ünite 3 — Yerleşim: Flexbox ve Grid.)

Adım 8: Gezinme çubuğunu Flexbox ile yatay dizin

❓ **Ne yapacağız?** stiller.css dosyanıza .site-nav ul seçicisi için display: flex kuralı-nı, aralarındaki boşluk için gap değerini yazacaksınız — şimdilik flex- direction: column ile (dikey), Bölüm 4'te bunu duyarlı hâle getireceksiniz.

📌 **Neden?** <nav> içindeki / listesi tek bir ekseninde (bir sıra menü ögesi) dizilir — bu, Flexbox'ın tam olarak çözmek için tasarlandığı problemdir: display: flex ile öğeler otomatik olarak yan yana ya da alt alta, aralarında eşit boşlukla dizilir; elle margin hesapla-manıza gerek kalmaz.

```

1  .site-nav ul {
2    list-style: none;
3    margin: 0;
4    padding: 0;
5    display: flex;
6    flex-direction: column;
7    gap: 0.5rem;
8  }
9
10 .site-nav a {
11   text-decoration: none;
12   font-weight: bold;

```

Adım 8 (devamı)

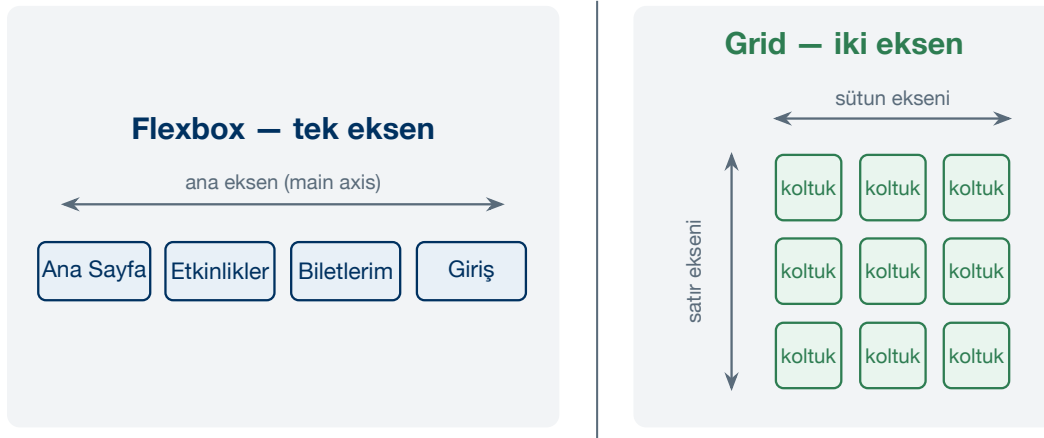
13 | }

👁 **Beklenen sonuç:** Menü bağlantıları arasında (dikey dizilimde) düzenli bir boşluk oluşur, alt çizgi kalkar ve bağlantılar kalın (bold) görünür.

Adım 9: Etkinlik kartları ızgarasını Grid ile oluşturun

❓ **Ne yapacağız?** stiller.css dosyanıza .etkinlik-listesi için display: grid ve tek sütunlu bir grid-template-columns kuralını, .etkinlik-karti için de kenarlık/köşe yuvarlama/iç boşluk/arka plan kurallarını ekleyeceksiniz.

📌 **Neden?** Kart listesi Bölüm 4'te ekran genişledikçe **çok sütunlu** bir ızgaraya dönüşecek — bu, hem satır hem sütun boyutunun birlikte tanımlı olduğu **iki boyutlu** bir yerleşim problemi; Flexbox tek eksenlidir ve satırlar sardığında sütun hizası bozulabilir, bu yüzden Grid tercih edilir.



Flexbox: sıra ya da sütun — tek yön. Grid: sıra VE sütun — birlikte.

```

1 .etkinlik-listesi {
2   display: grid;
3   grid-template-columns: 1fr;
4   gap: 1.5rem;
5   margin: 1rem 0;
6 }
7
8 .etkinlik-karti {
9   border: 1px solid var(--renk-gri);
10  border-radius: 0.5rem;
11  padding: 1rem;
12  background-color: var(--renk-acikgri);
13 }

```

👁 **Beklenen sonuç:** Kartlar artık açık gri arka planlı, yuvarlak köşeli, kenarlıklı kutular hâline gelir; aralarında düzenli bir dikey boşluk oluşur (henüz tek sütun hâlinde, çok sütunlu görünüm Bölüm 4'te gelecek).

Adım 10: etkinlik-detay.html sayfasını ve koltuk planı ızgarasını oluşturun

❓ **Ne yapacağız?** etkinlik-detay.html dosyanıza, index.html'de yazdığınız desenin aynısıyla (belge iskeleti + stiller.css bağlantısı + header/nav/main/footer) kendi sayfa iskeletinizi elle yazın (sekme başlığını "Bahar Konseri — Etkinlik Detayı" yapın); <main> içine

Adım 10 (devamı)

kısa bir başlık, bir <h2> ve bilet alma adımlarının sıralı bir listesini () ekleyin. Ardından aşağıdaki referans koltuk planı ızgarasını yazın.

i Neden? Koltuk planı, satır ve sütunun birlikte tanımlı olduğu klasik bir Grid problemidir: 3 satır × 4 sütun düzenli bir ızgara, repeat() ile tek satırda tanımlanabilir — bilet sistemine en doğal biçimde bağlanan yerleşim örneğidir.

```
1 <div class="koltuk-plani">
2   <div class="koltuk">A1</div>
3   <div class="koltuk">A2</div>
4   <div class="koltuk">A3</div>
5   <div class="koltuk">A4</div>
6   <div class="koltuk">B1</div>
7   <div class="koltuk">B2</div>
8   <div class="koltuk">B3</div>
9   <div class="koltuk">B4</div>
10  <div class="koltuk">C1</div>
11  <div class="koltuk">C2</div>
12  <div class="koltuk">C3</div>
13  <div class="koltuk">C4</div>
14 </div>
```

```
1 .koltuk-plani {
2   display: grid;
3   grid-template-columns: repeat(4, 1fr);
4   grid-template-rows: repeat(3, 1fr);
5   gap: 0.5rem;
6   max-width: 480px;
7   margin: 1rem 0;
8 }
9
10 .koltuk {
11   padding: 0.75rem;
12   text-align: center;
13   border: 1px solid var(--renk-yesil);
14   border-radius: 0.25rem;
15   background-color: var(--renk-acikmavi);
16 }
```

👁 Beklenen sonuç: Sayfada 3×4'lük düzenli bir ızgarada, açık mavi zeminli, yeşil kenarlıklı 12 koltuk kutusu (A1–C4) görünür. Koltuklar henüz tıklanamaz — yalnızca görsel bir ızgaradır; bu davranış Hf. 3'te eklenecektir.

Adım 11: Bilet formu iskeletini ekleyin

? Ne yapacağız? Koltuk planının altına bir <h2> ("Bilet Formu") ve <fieldset>/<legend> ile gruplanmış bir <form> yazacaksınız: ad soyad (metin), tarih, bilet adedi (sayı) girdileri ve bir koltuk sınıfı <select>'i — her <input> kendi <label>'ı ile for/id eşleşmesiyle bağlı olacak.

i Neden? label'ın for özniteliği ile input'un id'si eşleştiğinde, etikete tıklamak ilgili girdiyi odaklar — hem fare kullanımını kolaylaştırır hem de ekran okuyucunun "bu etiket hangi girdiye ait" bilgisini doğru okumasını sağlar (Ünite 4'te erişilebilirlik başlığı altında derinleşecek). action özniteliği kasıtlı olarak **yoktur** — form henüz gönderilmiyor, bu davranış Hf. 3'te DOM ile eklenecek.

Adım 11 (devamı)

```

1 <form>
2   <fieldset>
3     <legend>Bilet Bilgileri</legend>
4
5     <p>
6       <label for="ad-soyad">Ad Soyad</label>
7       <input type="text" id="ad-soyad" name="ad-soyad" required>
8     </p>
9
10    <p>
11      <label for="etkinlik-tarihi">Tarih</label>
12      <input type="date" id="etkinlik-tarihi" name="etkinlik-tarihi" value="2026-05-15" >
13    </p>
14
15    <p>
16      <label for="bilet-adedi">Bilet Adedi</label>
17      <input type="number" id="bilet-adedi" name="bilet-adedi" min="1" max="10" value="1">
18    </p>
19
20    <p>
21      <label for="koltuk-sinifi">Koltuk Sınıfı</label>
22      <select id="koltuk-sinifi" name="koltuk-sinifi">
23        <option value="ekonomi">Ekonomi</option>
24        <option value="orta">Orta Kat</option>
25        <option value="vip">VIP</option>
26      </select>
27    </p>
28
29    <button type="submit">Gönder</button>
30  </fieldset>
31 </form>

```

👁 **Beklenen sonuç:** Kenarlıklı bir çerçeve (fieldset) içinde “Bilet Bilgileri” başlığı (legend), altında etiket-girdi çiftleri ve bir “Gönder” düğmesi görünür. Düğmeye bastığınızda **hiçbir şey olmaz** — sayfa aynı kalır; bu beklenen bir davranıştır.

☑ Kontrol Noktası — devam etmeden doğrulayın

etkinlik-detay.html sayfanızda 3×4'lük koltuk ızgarasını ve etiket-girdi çiftleriyle bilet formunu, index.html'de ise nav menüsünün yatay boşluklu ve kart listesinin ızgara görünümünü doğruladıysanız, duyarlı tasarıma geçebilirsiniz.

4 Duyarlı Tasarım ve Erişilebilirlik

(bkz. Ders notu Hafta 2, Ünite 4 — Duyarlı Tasarım ve Erişilebilirlik Temelleri.)

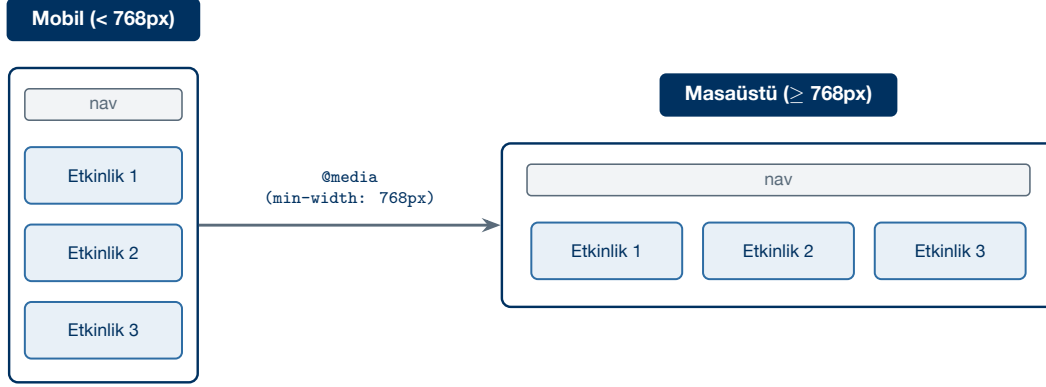
Adım 12: Medya sorgusu ekleyip iki kırılımda test edin

❓ **Ne yapacağız?** stiller.css dosyanızın en sonuna @media (min-width: 768px) bloğunu ekleyeceksiniz; bu blok içinde nav listesini yatay diziliminize (flex-direction: row) ve kart ızgarasını 3 sütuna (repeat(3, 1fr)) çevireceksiniz. Ardından tarayıcı penceresini el-

Adım 12 (devamı)

le daraltıp genişleterek (ya da geliştirici araçlarındaki cihaz simülatörüyle) iki kırılımı da gözlemleyeceksiniz.

i Neden? Mobil-önce yaklaşımda önce dar ekran için temel stiller yazılır (siz bunu zaten Bölüm 3'te yaptınız: dikey nav, tek sütunlu kart listesi); ekran genişledikçe `min-width` sorgularıyla **ek** stiller devreye girer — masaüstünde stil çıkarmak değil, **eklemek**. Bu yüzden medya sorgusu `max-width` değil `min-width` ile yazılır.



Mobil-önce: önce dar ekran için stil yazılır, sonra medya sorgusuyla geniş ekran stili EKLENİR.

```
1 @media (min-width: 768px) {
2   .site-nav ul {
3     flex-direction: row;
4     gap: 1.5rem;
5   }
6
7   .etkinlik-listesi {
8     grid-template-columns: repeat(3, 1fr);
9   }
10 }
```

👁 Beklenen sonuç: Pencere 768px'ten dar iken nav menüsü dikey, kart listesi tek sütun görünür (mobil kırılım). Pencereyi 768px eşiğinin üzerine genişlettiğinizde nav çubuğu yatay bir satıra, kart listesi 3 sütunlu bir ızgaraya aniden dönüşür — geçişin tam olarak 768px'te gerçekleştiğini gözlemleyin.

Adım 13: Kendiniz ekleyin: erişilebilirlik kontrol listesini uygulayın

? Ne yapacağız? Tamamladığınız iki sayfayı şu dört maddeyle kendiniz denetleyin: (1) her `<img>` etiketinin resmi anlatan dolu bir alt metni var mı? (2) her `<label>`'ın for'u ilgili `<input>`'un id'siyle birebir eşleşiyor mu — etikete tıklayınca girdi odaklanıyor mu? (3) Tab tuşuna art arda basarak sayfada gezinin — odaklanan öğenin etrafında (kehribar renkli) belirgin bir çerçeve görüyor musunuz? (4) metin ile zemin arasındaki renk kontrastı — özellikle header/ footer üzerindeki beyaz metin — gözle rahatça okunuyor mu? Eksik bulduğunuz maddeyi kendi dosyanızda düzeltin.

i Neden? Semantik landmark öğeleri (Ünite 1) ekran okuyuculara sayfanın bölgelerini bildirir ama tek başına yeterli değildir — resimlerin alt metni, etiket-girdi eşleşmesi, görünür klavye odağı ve yeterli kontrast olmadan sayfa yine bazı kullanıcılar için kullanılamaz kalır. Bu dört madde, bir sayfanın “erişilebilir mi” sorusuna hızlıca cevap verdiğiniz asgari bir kontrol listesidir.

👁 Beklenen sonuç: Dört maddenin tamamını işaretleyebiliyorsanız (özellikle `stilller.css`'in başındaki `:focus` kuralı sayesinde Tab ile gezinirken kehribar rengi

Adım 13 (devamı)

bir çerçeve gördüyseniz), sayfanız asgari erişilebilirlik denetiminden geçmiş demektir.

İpucu

Çoğu tarayıcının geliştirici araçlarında bir “Erişilebilirlik” (*Accessibility*) sekmesi bulunur; bu sekme eksik alt metni ya da düşük kontrast gibi sorunları otomatik olarak listeler — elle kontrolünüzü tamamladıktan sonra bir de bu araçla doğrulayın.

✓ Kontrol Noktası — devam etmeden doğrulayın

Pencereyi daraltıp genişlettiğinizde 768px eşiğinde nav ve kart listesinin dönüştüğünü, ayrıca erişilebilirlik kontrol listesinin dört maddesini de işaretlediğinizi doğruladıysanız, bu haftanın statik etkinlik sayfası tamamlanmış demektir.

✖ Sorun Giderme

Belirti	Olası neden	Çözüm
Sayfa açılırken tarayıcı “dosya bulunamadı” hatası veriyor CSS hiç uygulanmıyor, sayfa düz metin gibi görünüyor	Yanlış klasörden ya da yanlış dosya adıyla açmaya çalışıyorsunuz stiller.css dosyası index.html’e bağlanmamış ya da href yazımı hatalı	web-lab klasöründeki index.html dosyasını açtığınızı kontrol edin <link rel="stylesheet" href="stiller.css"> satırının <head> içinde ve dosya adının birebir doğru olduğunu kontrol edin
Kartlar/form beklenenden geniş görünüyor, taşıyor	box-sizing: border-box kuralı eksik ya da dosyanın sonuna eklenmiş	Kutu modeli sıfırlama kuralının stiller.css’in en başında olduğunu doğrulayın
768px eşiğinde nav/kart yerleşimi hiç değişmiyor	Tarayıcı penceresi tam ekran genişlikte tutuluyor, eşik hiç aşılmıyor	Pencereyi elle daraltın ya da geliştirici araçlarındaki cihaz simülatörünü kullanın
Etikete tıklayınca ilgili girdi odaklanmıyor	label’in for değeri ile input’un id’si birebir eşleşmiyor (yazım hatası)	İki değeri — büyük/küçük harf dahil — karşılaştırıp düzeltin
Türkçe karakterler (ğ, ş, ı) tarayıcıda bozuk görünüyor	<meta charset="UTF-8"> eksik ya da dosya farklı bir kodlamayla kaydedilmiş	<head>’in ilk satırları arasında charset="UTF-8" olduğunu ve dosyanın UTF-8 kaydedildiğini doğrulayın

5 • İzleme ➡ Aktarım — Başka Nerede?

Bugün elle yazdığınız header/nav/main/footer iskeleti, kart ızgarası, koltuk planı ve bilet formu — Etkinlik/Bilet Sisteminin statik etkinlik sayfasının omurgası olarak dönem boyunca sizinle kalacak. Hf. 3’te bu sayfaya JavaScript’in dil modelini öğrenip TypeScript ile ilk davranışı ekleyeceksiniz: bir koltuğa tıklayınca rengi değişecek, bilet formu artık gerçekten gönderilecek — HTML/CSS yapınız **değişmeden** kalacak, yalnızca üzerine kod eklenecek. Hf. 4’te bugün <article> olarak sınırlarını çizdiğiniz her etkinlik kartı ayrı bir React bileşenine bölünecek; koltuk planınız Hf. 5’te bir dizi üzerinden map ile dinamik olarak üretilecek. box-sizing:

border-box tercihiniz Hf. 4'ten itibaren React bileşen stillerinde de aynen sürecektir. Son olarak, bugün 768px eşiğiyle kurduğunuz duyarlı tasarım, Hf. 14'te Docker ile canlıya aldığınız uygulamaya telefon, tablet ve masaüstünden gelen gerçek kullanıcı trafiğinde önemini koruyacak.